

1. Executive Summary

This report documents a password recovery lab involving an encrypted ZIP archive. By utilizing industry-standard cryptographic tools in a Kali Linux environment, a password-protected file was successfully audited, its hash exfiltrated, and the plaintext password recovered. This exercise highlights the vulnerabilities associated with weak password policies and legacy encryption formats that are susceptible to high-speed recovery attacks.

2. Technical Methodology & Walkthrough

The assessment involved a multi-stage process: creating a secure asset, converting its protection into a crackable format (hashing), and performing a dictionary attack.

Stage 1: Target Preparation

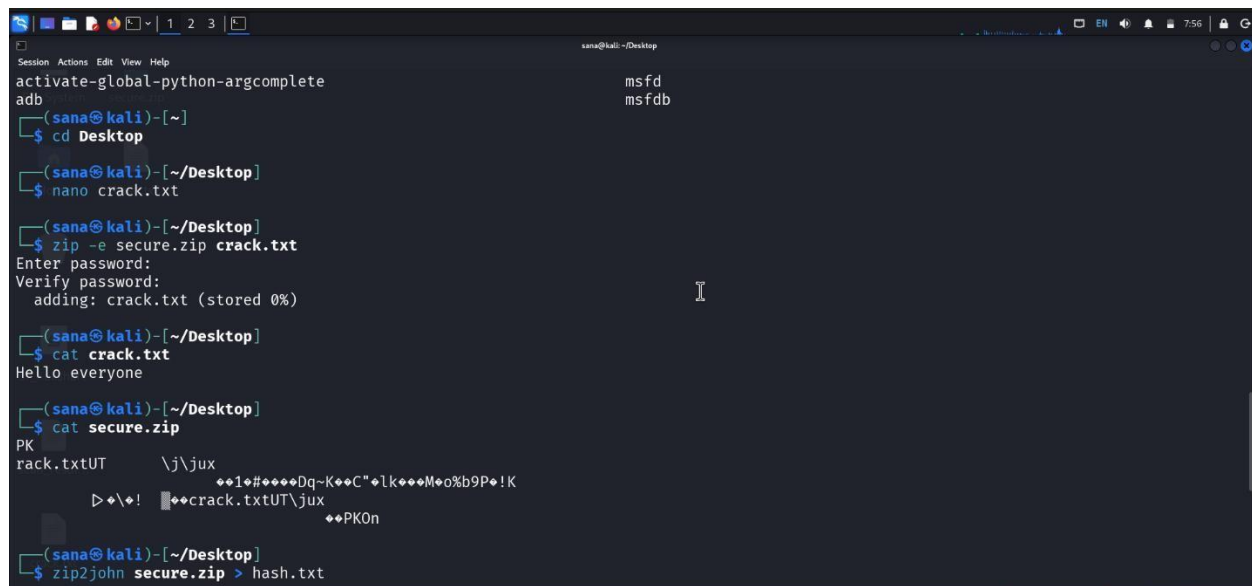
The initial file, `crack.txt`, was created containing the string "Hello everyone" and encrypted into a ZIP archive named `secure.zip`.

- **Action:** The file was encrypted using the standard Linux `zip` utility with the `-e` flag.
- **Command:** `zip -e secure.zip crack.txt`.

Stage 2: Hash Extraction

To prepare for the cracking process, the cryptographic metadata was extracted from the archive to create a format compatible with cracking tools.

- **Action:** The `zip2john` utility was used to generate a standalone hash file.
- **Command:** `zip2john secure.zip > hash.txt`.
- **Observation:** The tool identified the encryption as **PKZIP Encr**, a legacy format



```
sana@kali:~/Desktop
Session Actions Edit View Help
activate-global-python-argcomplete
adb
msfd
msfdb

(sana@kali)~[~]
$ cd Desktop

(sana@kali)~/Desktop
$ nano crack.txt

(sana@kali)~/Desktop
$ zip -e secure.zip crack.txt
Enter password:
Verify password:
adding: crack.txt (stored 0%)

(sana@kali)~/Desktop
$ cat crack.txt
Hello everyone

(sana@kali)~/Desktop
$ cat secure.zip
PK
rack.txtUT      \j\jux
               ♦♦1♦#♦♦♦♦Dq~K♦♦C"♦lk♦♦♦M♦o%b9P♦!K
               ▷♦\♦!  ♦♦crack.txtUT\jux
               ♦♦PK0n

(sana@kali)~/Desktop
$ zip2john secure.zip > hash.txt
```

Stage 3: Password Cracking

A dictionary attack was launched against the extracted hash to recover the plaintext password.

- **Action:** The John the Ripper tool was used to compare the hash against a wordlist.
- **Command:** john hash.txt.

```
(sana@kali)-[~/Desktop]
$ cat crack.txt
Hello everyone

(sana@kali)-[~/Desktop]
$ cat secure.zip
PK
crack.txtUT      \j\jux
                ++1e#++++Dq~K++C"•lk+++M+o%b9P+!K
                ▷\+!  ++crack.txtUT\jux
                ++PK0n

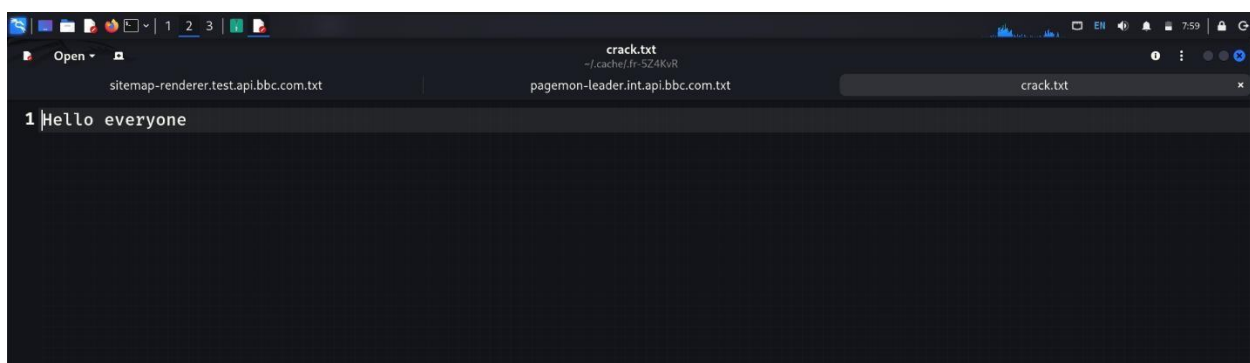
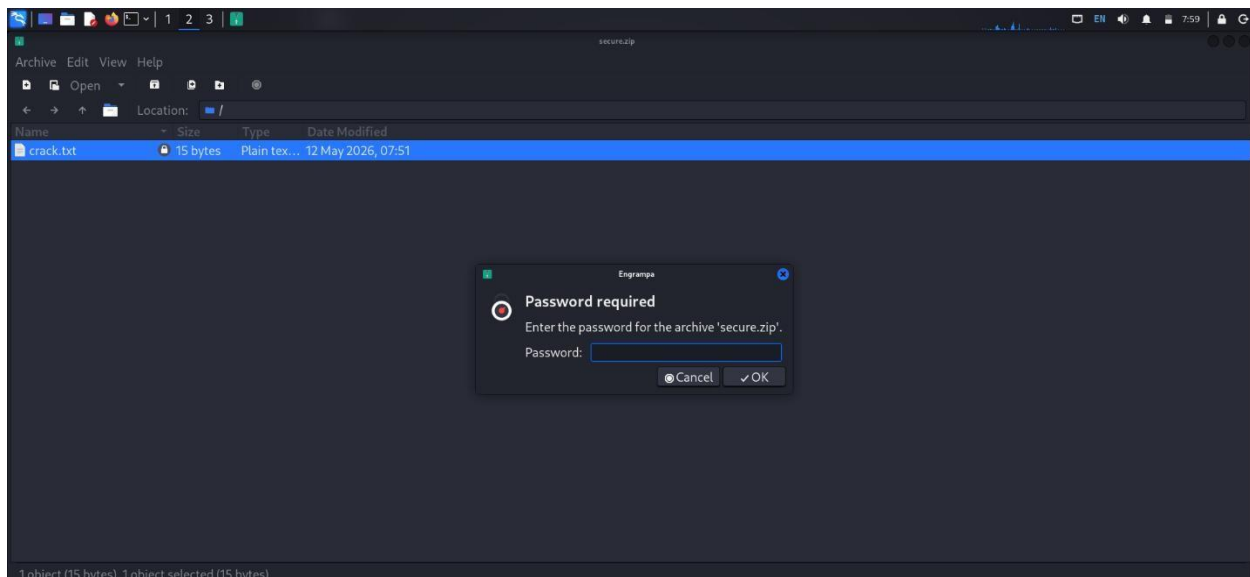
(sana@kali)-[~/Desktop]
$ zip2john secure.zip > hash.txt
Created directory: /home/sana/.john
ver 1.0 efh 5455 efh 7875 secure.zip/crack.txt PKZIP Encr: 2b chk, TS_chk, cmplen=27, decmplen=15, crc=21811012 ts=3E7C cs=3e7c type=0

(sana@kali)-[~/Desktop]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 6 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Admin123 (secure.zip/crack.txt)
1g 00:00:00:00 DONE (2026-05-12 07:55) 4.3+7g/s 9456Kp/s 9456Kc/s 9456Kc/s Angelgirl2..962792642
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Stage 4: Verification of Access

The recovered password was used to decrypt the archive and verify the integrity of the original file.

- **Action:** The archive was opened using the recovered password to access the crack.txt file.



3. Diagnostic Analysis & Impact

The successful recovery of the password demonstrates several critical security risks:

- **Nullified Confidentiality:** Unauthorized users can easily access sensitive documents when weak passwords are used.
- **Lateral Movement Risks:** If a weak password like "Admin123" is reused across other corporate systems, the entire network's security is compromised.
- **Data Integrity Concerns:** Once the archive is unlocked, an attacker can modify contents without the owner's knowledge.

4. Proposed Security Solutions

To mitigate the risks identified in this walkthrough, the following measures are recommended:

- **Implement Stronger Encryption:** Move away from legacy PKZIP encryption and utilize **AES-256 encryption**, supported by modern tools like 7-Zip.
- **Enforce Password Complexity:** Implement policies requiring passwords to be at least 12 characters long, including a mix of special characters while avoiding common words like "Admin".
- **Key-Based Authentication:** For highly sensitive files, utilize **GPG encryption** with public/private key pairs instead of simple symmetric passwords.